

핸즈온 랩: Microsoft Fabric에서 배포된 모델을 사용하여 배치 예측 생성하기

이 랩에서는 기계 학습 모델을 사용하여 당뇨병의 정량적 측정치를 예측합니다.

이 랩을 완료하면 예측 생성 및 결과 시각화에 대한 실무 경험을 얻게 됩니다.

이 실습을 완료하는 데 약 **25분**이 소요됩니다.

참고: 이 실습을 완료하려면 [Microsoft Fabric 평가판](#)이 필요합니다.

Workspace 만들기

Fabric에서 데이터 작업을 시작하기 전에 Fabric 평가판이 활성화된 Workspace를 만들어야 합니다.

- 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) `https://app.fabric.microsoft.com/home?experience=fabric` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
- 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
- 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial*, *Premium*, 또는 *Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
- 새 Workspace가 열리면 처음에는 비어 있어야 합니다.

Notebook 만들기

이 실습에서는 모델을 학습시키고 사용하기 위해 **Notebook**을 사용합니다.

- 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Science* 섹션에서 **Notebook**을 선택하고, 원하는 고유한 이름을 지정합니다.
- 첫 번째 셀을 Markdown 셀로 변환하고 다음 텍스트를 입력합니다.

```
# Train and use a machine learning model
```

머신러닝 모델 학습하기

먼저, 당뇨병 환자의 관심 반응(기준선 1년 후의 질병 진행 정량적 측정치)을 예측하기 위해 *회귀(regression)* 알고리즘을 사용하는 머신러닝 모델을 학습시키겠습니다.

개념 설명: 모델 서명 (Model Signature)

모델 서명은 기계 학습 모델의 입력(input)과 출력(output) 스키마를 명시적으로 정의하는 것입니다. 즉, 모델이 어떤 이름과 데이터 타입의 열들을 입력으로 받고, 어떤 이름과 데이터 타입의 열을 출력으로 반환하는지를 명확히 하는 '계약서'와 같습니다. 모델 서명을 정의하면, 모델을 사용할 때 입력 데이터가 올바른 형식인지 쉽게 검증할 수 있으며, 예측 함수의 출력을 예측 가능하게 만들어 모델의 배포와 사용을 더 안정적이고 편리하게 만들어 줍니다.

- Notebook에 새 코드 셀을 추가하고, 데이터를 로드 및 준비하고 모델을 학습시키는 다음 코드를 입력합니다.

```
import pandas as pd
```

```

import mlflow
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from mlflow.models.signature import ModelSignature
from mlflow.types.schema import Schema, ColSpec

# Get the data
# (Azure Open Datasets에서 당뇨병 데이터 로드)
blob_account_name = "azureopendatastorage"
blob_container_name = "mlsamples"
blob_relative_path = "diabetes"
blob_sas_token = r""
wasbs_path = f"wasbs://%s@%s.blob.core.windows.net/%s" % (blob_container_name,
blob_account_name, blob_relative_path)
spark.conf.set("fs.azure.sas.%s.%s.blob.core.windows.net" % (blob_container_name,
blob_account_name), blob_sas_token)
df = spark.read.parquet(wasbs_path).toPandas()

# Split the features and label for training
# (학습을 위해 특징과 레이블 분리)
X, y = df[['AGE', 'SEX', 'BMI', 'BP', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6']].values,
df['Y'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,
random_state=0)

# Train the model in an MLflow experiment
# (MLflow 실험에서 모델 학습)
experiment_name = "experiment-diabetes"
mlflow.set_experiment(experiment_name)
with mlflow.start_run():
    mlflow.autolog(log_models=False) # 모델은 수동으로 로그할 것이므로 자동 로깅은 비활성화
    model = DecisionTreeRegressor(max_depth=5)
    model.fit(X_train, y_train)

# Define the model signature
# (모델 서명 정의)
input_schema = Schema([
    ColSpec("integer", "AGE"), ColSpec("integer", "SEX"),
    ColSpec("double", "BMI"), ColSpec("double", "BP"),
    ColSpec("integer", "S1"), ColSpec("double", "S2"),
    ColSpec("double", "S3"), ColSpec("double", "S4"),
    ColSpec("double", "S5"), ColSpec("integer", "S6"),
])
output_schema = Schema([ColSpec("integer")])
signature = ModelSignature(inputs=input_schema, outputs=output_schema)

# Log the model
# (서명과 함께 모델 로그)
mlflow.sklearn.log_model(model, "model", signature=signature)

```

2. 셀 왼쪽의 ▶ **Run cell** 버튼을 사용하여 코드를 실행합니다.

3. 새 코드 셀을 추가하고, 이전 셀의 실험에서 학습된 모델을 등록하는 다음 코드를 입력합니다.

```
# Get the most recent experiment run
# (가장 최근의 실험 실행 가져오기)
exp = mlflow.get_experiment_by_name(experiment_name)
last_run = mlflow.search_runs(exp.experiment_id, order_by=["start_time DESC"],
max_results=1)
last_run_id = last_run.iloc[0]["run_id"]

# Register the model that was trained in that run
# (해당 실행에서 학습된 모델 등록)
print("Registering the model from run :", last_run_id)
model_uri = "runs:/{}/model".format(last_run_id)
mv = mlflow.register_model(model_uri, "diabetes-model")
print("Name: {}".format(mv.name))
print("Version: {}".format(mv.version))
```

코드 설명:

- `mlflow.get_experiment_by_name(...)`: 이름으로 실험을 찾습니다. 반환값은 `experiment` 객체. 이 객체에는 `experiment_id`, `name`, `artifact_location` 등의 정보가 들어있습니다.
- `mlflow.search_runs(...)`: 해당 실험 내에서 가장 최근에 실행된(`start_time` 기준 내림차순 정렬) run 1개를 찾습니다.
- `last_run_id = last_run.iloc[0]["run_id"]`: 위에서 가져온 결과는 Pandas DataFrame입니다. `iloc[0]`은 첫 번째 row(가장 최근 run)를 가져오고, 그 중 `run_id` 값을 추출합니다. 이 `run_id`는 MLflow에서 해당 실행(run)을 고유하게 식별하는 데 사용됩니다.
- `model_uri`는 MLflow에서 모델의 경로를 지정하는 URI입니다.
 - `"runs:<run_id>/model"` 형식이며,
 - 해당 run에서 저장된 아티팩트 중 `"model"`이라는 이름의 디렉토리를 가리킵니다.
 - 이 경로는 `mlflow.<flavor>.log_model()`로 저장한 모델의 위치와 일치해야 합니다.
- `mlflow.register_model(...)`: 찾은 run에서 저장된 모델(`model_uri`)을 가져와 `diabetes-model`이라는 이름으로 Fabric Model Registry에 공식적으로 등록합니다. 이제 이 모델은 버전 관리가 가능하며, 다른 사용자와 공유하고 배포할 수 있습니다.

```

1 # Get the most recent experiment run
2 # (가장 최근의 실험 실행 가져오기)
3 exp = mlflow.get_experiment_by_name(experiment_name)
4 last_run = mlflow.search_runs(exp.experiment_id, order_by=["start_time DESC"], max_results=1)
5 last_run_id = last_run.iloc[0]["run_id"]
6
7 # Register the model that was trained in that run
8 # (해당 실행에서 학습된 모델 등록)
9 print("Registering the model from run :", last_run_id)
10 model_uri = "runs:/{}/model".format(last_run_id)
11 mv = mlflow.register_model(model_uri, "diabetes-model")
12 print("Name: {}".format(mv.name))
13 print("Version: {}".format(mv.version))

```

✓ 16 sec - Command executed in 16 sec 821 ms by 이정환 on 2:46:07 AM, 7/29/25

>  Log

Registering the model from run : 33c7c77c-77db-48c1-b417-17eaf9d1e186
 Successfully registered model 'diabetes-model'.
 Created version '1' of model 'diabetes-model'.

이제 모델이 **diabetes-model**이라는 이름으로 Workspace에 저장되었습니다. 선택적으로, Workspace에서 찾아보기 기능을 사용하여 모델을 찾고 UI를 통해 탐색할 수 있습니다.

Lakehouse에 테스트 데이터 세트 만들기

모델을 사용하려면, 당뇨병 진단을 예측해야 할 환자 세부 정보 데이터 세트가 필요합니다. 이 데이터 세트를 Microsoft Fabric Lakehouse의 테이블로 생성합니다.

1. Notebook 편집기의 왼쪽 **Explorer** 창에서 **+ Data sources**를 선택하여 Lakehouse를 추가합니다.

No data sources added

Add data items ▾

New lakehouse

Existing data sources

2. **New lakehouse**를 선택하고 **Add**를 선택한 다음, 원하는 유효한 이름으로 새 **Lakehouse**를 만듭니다.
3. 현재 세션을 중지하라는 메시지가 나타나면 **Stop now**를 선택하여 Notebook을 다시 시작합니다.
4. Lakehouse가 생성되고 Notebook에 연결되면, 새 코드 셀을 추가하고 다음 코드를 실행하여 데이터 세트를 만들고 Lakehouse의 테이블에 저장합니다.

```

from pyspark.sql.types import IntegerType, DoubleType

# Create a new dataframe with patient data
data = [

```

```

(62, 2, 33.7, 101.0, 157, 93.2, 38.0, 4.0, 4.8598, 87),
(50, 1, 22.7, 87.0, 183, 103.2, 70.0, 3.0, 3.8918, 69),
(76, 2, 32.0, 93.0, 156, 93.6, 41.0, 4.0, 4.6728, 85),
(25, 1, 26.6, 84.0, 198, 131.4, 40.0, 5.0, 4.8903, 89),
(53, 1, 23.0, 101.0, 192, 125.4, 52.0, 4.0, 4.2905, 80),
(24, 1, 23.7, 89.0, 139, 64.8, 61.0, 2.0, 4.1897, 68),
(38, 2, 22.0, 90.0, 160, 99.6, 50.0, 3.0, 3.9512, 82),
(69, 2, 27.5, 114.0, 255, 185.0, 56.0, 5.0, 4.2485, 92),
(63, 2, 33.7, 83.0, 179, 119.4, 42.0, 4.0, 4.4773, 94),
(30, 1, 30.0, 85.0, 180, 93.4, 43.0, 4.0, 5.3845, 88)
]
columns = ['AGE', 'SEX', 'BMI', 'BP', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6']
df = spark.createDataFrame(data, schema=columns)

# Convert data types to match the model input schema
# (모델 입력 스키마와 일치하도록 데이터 유형 변환)
df = df.withColumn("AGE", df["AGE"].cast(IntegerType()))
df = df.withColumn("SEX", df["SEX"].cast(IntegerType()))
df = df.withColumn("BMI", df["BMI"].cast(DoubleType()))
df = df.withColumn("BP", df["BP"].cast(DoubleType()))
df = df.withColumn("S1", df["S1"].cast(IntegerType()))
df = df.withColumn("S2", df["S2"].cast(DoubleType()))
df = df.withColumn("S3", df["S3"].cast(DoubleType()))
df = df.withColumn("S4", df["S4"].cast(DoubleType()))
df = df.withColumn("S5", df["S5"].cast(DoubleType()))
df = df.withColumn("S6", df["S6"].cast(IntegerType()))
df = df.withColumn("S6", df["S6"].cast(IntegerType()))

# Save the data in a delta table
table_name = "diabetes_test"
df.write.format("delta").mode("overwrite").saveAsTable(table_name)
print(f"Spark dataframe saved to delta table: {table_name}")

```

5. 코드가 완료되면, **Lakehouse explorer** 창의 **Tables** 옆에 있는 ...를 선택하고 **Refresh**를 선택합니다.
diabetes_test 테이블이 나타나야 합니다.

6. 왼쪽 창에서 **diabetes_test** 테이블을 확장하여 포함된 모든 필드를 확인합니다.

모델을 적용하여 예측 생성하기

개념 설명: MLFlowTransformer (PREDICT 함수)

MLFlowTransformer는 Fabric Notebook 환경에서 MLflow에 등록된 모델을 사용하여 배치 예측을 수행하는 매우 편리한 방법입니다. 이는 내부적으로 T-SQL의 **PREDICT** 함수와 유사하게 작동합니다. **MLFlowTransformer**를 사용하면 복잡한 코드 없이 모델 이름과 버전을 지정하고, 입력 및 출력 열 이름만 설정하면 Spark DataFrame에 대해 대규모 예측을 쉽게 수행할 수 있습니다.

핸즈온의 의미: 이 단계는 데이터 과학 워크플로우의 마지막 단계인 **추론(Inference)** 또는 **스코어링(Scoring)**을 직접 수행하는 과정입니다. 앞서 학습하고 등록한 모델을 불러와, 새로운 데이터(여기서는 **diabetes_test** 테이블)에 적용하여 예측값을 생성하고, 그 결과를 다시 테이블에 저장하는 전체 흐름을 경험합니다.

1. 새 코드 셀을 추가하고 다음 코드를 실행합니다.

```
import mlflow
from synapse.ml.predict import MLFlowTransformer

## Read the patient features data
## (환자 특징 데이터 읽기)
df_test = spark.read.format("delta").load(f"Tables/{table_name}")

# Use the model to generate diabetes predictions for each row
# (모델을 사용하여 각 행에 대한 당뇨병 예측 생성)
model = MLFlowTransformer(
    inputCols=["AGE", "SEX", "BMI", "BP", "S1", "S2", "S3", "S4", "S5", "S6"],
    outputCol="predictions",
    modelName="diabetes-model",
    modelVersion=1)
df_test = model.transform(df)

# Save the results (the original features PLUS the prediction)
# (결과 저장 - 원본 특징 + 예측값)
df_test.write.format('delta').mode("overwrite").option("mergeSchema",
"true").saveAsTable(table_name)
```

코드 설명:

- `MLFlowTransformer(...): diabetes-model` 버전 1을 사용하여 예측을 수행할 변환기를 생성합니다.
 - `inputCols`: 모델에 입력으로 사용할 열 목록을 지정합니다.
 - `outputCol`: 예측 결과가 저장될 새 열의 이름을 `predictions` 로 지정합니다.
- `model.transform(df)`: `MLFlowTransformer` 를 사용하여 `df` 데이터프레임의 각 행에 대해 예측을 수행합니다.
- `option("mergeSchema", "true")`: 테이블을 덮어쓸 때, 기존 스키마에 `predictions` 라는 새 열이 추가되는 스키마 변경을 허용하는 옵션입니다.

```

1 import mlflow
2 from synapse.ml.predict import MLFlowTransformer
3
4 ## Read the patient features data
5 ## (환자 특징 데이터 읽기)
6 df_test = spark.read.format("delta").load(f"Tables/{table_name}")
7
8 # Use the model to generate diabetes predictions for each row
9 # (모델을 사용하여 각 행에 대한 당뇨병 예측 생성)
10 model = MLFlowTransformer(
11     inputCols=["AGE", "SEX", "BMI", "BP", "S1", "S2", "S3", "S4", "S5", "S6"],
12     outputCol="predictions",
13     modelName="diabetes-model",
14     modelVersion=1)
15 df_test = model.transform(df)
16
17 # Save the results (the original features PLUS the prediction)
18 # (결과 저장 - 원본 특징 + 예측값)
19 df_test.write.format('delta').mode("overwrite").option("mergeSchema", "true").saveAsTable(table_name)

```

[*]

✖ 22 sec - Running

>

Spark jobs In progress (4)

Resources
 Log

...

Downloading artifacts: 100% 9/9 [00:00<00:00, 4.59it/s]

Downloading artifacts: 100% 9/9 [00:00<00:00, 25.83it/s]

/opt/spark/python/lib/pyspark.zip/pyspark/sql/pandas/functions.py:407: UserWarning: In Python 3.6+ and Spark 3.0+, it is preferred specifying pandas UDF type which will be deprecated in the future releases. See SPARK-28264 for more details.

2. 코드가 완료된 후, **Lakehouse explorer** 창의 **diabetes_test** 테이블 옆에 있는 ...를 선택하고 **Refresh**를 선택합니다. 새 필드 **predictions**가 추가된 것을 확인합니다.

3. 새 코드 셀을 추가하고 **diabetes_test** 테이블을 그 위로 끌어다 놓습니다. 테이블 내용을 보기 위한 필요한 코드가 자동으로 나타납니다. 셀을 실행하여 데이터를 표시합니다. 이제 원본 데이터 옆에 모델이 예측한 값이 함께 있는 것을 볼 수 있습니다.

```
1 df = spark.sql("SELECT * FROM LHds.diabetes_test LIMIT 1000")
2 display(df)
```

✓ 5 sec - Command executed in 4 sec 978 ms by 이정환 on 3:10:03 AM, 7/29/25

> Spark jobs (7 of 7 succeeded) Resources

Table + New chart

Table view

	123 AGE	123 SEX	12 BMI	12 BP	123 S1	12 S2	12 S3	12 S4	12 S5	123 S6	123 predictions
1	25	1	26.6	84.0	198	131.4	40.0	5.0	4.8903	89	166
2	53	1	23.0	101.0	192	125.4	52.0	4.0	4.2905	80	122
3	63	2	33.7	83.0	179	119.4	42.0	4.0	4.4773	94	143
4	30	1	30.0	85.0	180	93.4	43.0	4.0	5.3845	88	255
5	62	2	33.7	101.0	157	93.2	38.0	4.0	4.8598	87	143
6	69	2	27.5	114.0	255	185.0	56.0	5.0	4.2485	92	52
7	76	2	32.0	93.0	156	93.6	41.0	4.0	4.6728	85	143
8	50	1	22.7	87.0	183	103.2	70.0	3.0	3.8918	69	76
9	38	2	22.0	90.0	160	99.6	50.0	3.0	3.9512	82	75
10	24	1	23.7	89.0	139	64.8	61.0	2.0	4.1897	68	76

리소스 정리

- 이 실습에서는 모델을 사용하여 배치 예측을 생성했습니다.
- Notebook 탐색을 마쳤다면 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.
- 1. 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
 - 2. 툴바의 ... 메뉴에서 **Workspace settings**를 선택합니다.
 - 3. **General** 섹션에서 **Remove this workspace**를 선택합니다.